

Shashank Chandavarkar

College Station, TX — shashank3087@gmail.com — github.com/ShashankC10 — linkedin.com/in/shashank-chandavarkar

Education

Texas A&M University, College Station, TX

Aug. 2025 – May 2027 (Expected)

Master of Science in Computer Science

GPA: 4.0/4.0

JSS Science and Technology University, Mysore, India

Aug. 2018 – Jul. 2022

Bachelor of Engineering in Computer Science

GPA: 3.8/4.0

Experience

Sixt Rent a Car

SOFTWARE DEVELOPMENT ENGINEER

Jul. 2022 – Jun. 2025

- Owned IoT lifecycle service powering **300k+ vehicles**, handling device activation/deactivation via **Kafka-driven Spring Boot pipelines** with **KIE/DRL rule evaluation**.
- Migrated vehicle search from a Go-based wrapper endpoint to a native **Java Spring Boot service**, eliminating an extra network hop and reducing **~100k network calls per day**.
- Migrated operations app endpoints from REST to **gRPC**, leveraging binary serialization and HTTP/2 to achieve **~15% reduction in latency**.
- Built a **Python/Flask microservice** to recommend relocating free-floating vehicles, improving fleet utilization by **~40%** on pilot cohorts.
- Built a **Go-based vehicle search service** that ingested and aggregated real-time Share fleet events (location, fuel, mileage) into an **Elasticsearch index** for low-latency querying.
- Powered the **FleetOps dashboard** with real-time vehicle positioning and status data, enabling operations teams to monitor fleet health and create service orders across active markets.
- Built a return-time approval feature that cut incorrect invoicing by **~25%** and reduced manual overrides at scale.
- Served on-call for mobility services, added **distributed tracing and alerting**, reducing false positives and improving MTTR for fleet incidents.

Skills

Languages: Java, Go, Python, C++, JavaScript, TypeScript

Frameworks & Libraries: Spring Boot, FastAPI, Angular, gRPC, JPA, Hibernate

AI & LLMs: LangChain, LangGraph, Ollama, PyTorch, scikit-learn, Pandas, OpenRouter

Databases & Search: PostgreSQL, MySQL, MongoDB, Redis, Elasticsearch, OpenSearch

Infrastructure & DevOps: AWS, Docker, Kubernetes, Kafka, Jenkins, Linux, Git

Observability: Prometheus, Grafana, Instana, OpenSearch Dashboards

Projects

Clinical Trial Search Engine

GitHub

- Built a hybrid search engine combining **BM25 (OpenSearch)** and **dense semantic retrieval (PubMedBERT)** using **Reciprocal Rank Fusion (RRF)**, indexing **560k+ clinical trials**.
- Designed a large-scale ingestion pipeline to scrape, normalize, and index clinical trial data from ClinicalTrials.gov into **PostgreSQL and OpenSearch**.
- Integrated **FAISS-based vector search** to re-rank top candidates per query and implemented an AI-driven feasibility scoring engine with cached eligibility parsing.
- Achieved **sub-2s query latency** and strong retrieval quality on the **TREC 2021 Clinical Trials** benchmark (**0.48 MRR@10, 70% Hit Rate@10**), containerized using Docker.

Prometheus AI Agent

GitHub

- Built a **LangChain ReAct agent** that translates natural language questions into **PromQL**, executes queries against **Prometheus**, and returns incident-style root-cause summaries, eliminating the need to write PromQL manually.
- Implemented statistical anomaly detection over **Prometheus time-series data** using z-score analysis with trend classification (increasing / stable / decreasing) and spike/dip identification.
- Engineered a **provider-agnostic LLM backend** supporting **Anthropic Claude** and **Ollama-served models** via a config-driven switch, validated with **Qwen 3.5 397B** (BF16, 262K context).

AI-Powered Regulatory Document Classifier

GitHub

- Developed a **multi-modal AI system** combining text parsing, **BLIP-based image captioning**, and **LLM-driven reasoning** via OpenRouter API.
- Built a dynamic prompt library enabling **real-time compliance rule updates**, explainable classifications, and human in the loop feedback, containerized for scalable deployment.

Task Management & Analytics Platform

GitHub — GitHub

- Designed a **two-service Spring Boot system** with REST APIs, JPA persistence, and a **state-machine-driven task lifecycle** (PENDING → IN_PROGRESS → DONE).
- Published task lifecycle events to **Kafka** and consumed them in a separate analytics service to compute throughput, latency, and state-transition metrics, indexing results into **OpenSearch** for real-time dashboards.
- Containerized both services with **Docker Compose** and added **CI pipelines** via GitHub Actions for automated builds and testing.